Physiological Control Systems Projects

Michelle L. Wu

Fall 2025
Physiological Control Systems
Taught by: Dr. Michael Khoo

Contents

1 Project A

Group 9

Present your simulation model and results as a PDF document. In this report,

- address the above questions in the sequence in which they were asked.
- include plots of the various scenarios (it would be useful to employ the same ordinate- and time-axis scales, if possible, for easy comparison across cases), the parameter values that were changed for each scenario, and of course, your responses to the questions posed to you.
- include diagram(s) of your revised Simulink model(s), the Simulink codes ("`.slx`" file), the Matlab parameter file, and any other relevant files in one zipped folder.

Please note that LaTeX won't compile diagram(s) of the revised Simulink model(s), the Simulink codes ("`.slx`" file), the Matlab parameter file, and any other relevant files. They have been included in the zipped folder.

1.1 Part I

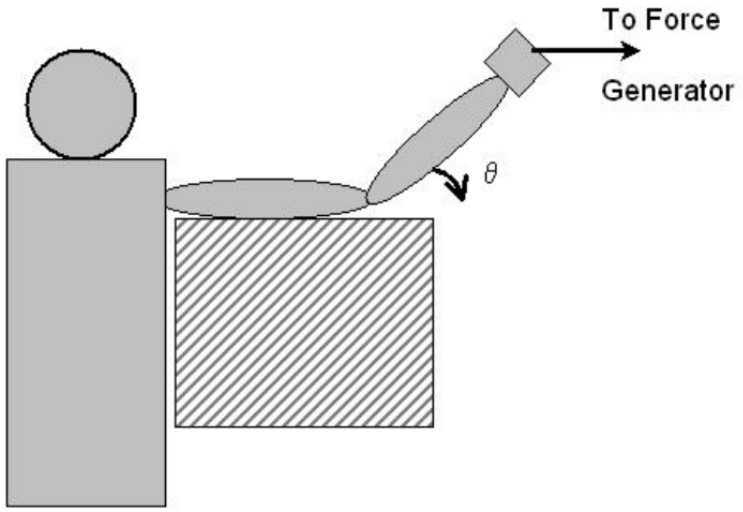

Figure 1

The experiment depicted in Figure 1 was discussed in class: the subject is seated comfortably, and his shoulder and elbow are held by adjustable supports so that the upper arm remains in a fixed horizontal position throughout the test. The subject's forearm is allowed to move only in the vertical plane. Prior to the start of the experiment, he maintains a constant torque to balance the external torque produced by a force generator that is coupled to his forearm. Thus, initially, the angle between the forearm and upper arm is 135°. Assume that you are able to make the force generator produce small changes $M_x(t)$ in external torque above and below the original baseline level. Although the subject is told to try his best

to maintain his forearm at the original baseline angle, the changes in external torque are expected to lead to small changes in angular position, $\theta(t)$ of the forearm about the elbow joint. Note that $\theta(t)$ is measured relative to the initial angle between forearm and upper arm of 135°; as shown in Figure 1, $\theta(t)$ is positive when the forearm rotates clockwise.

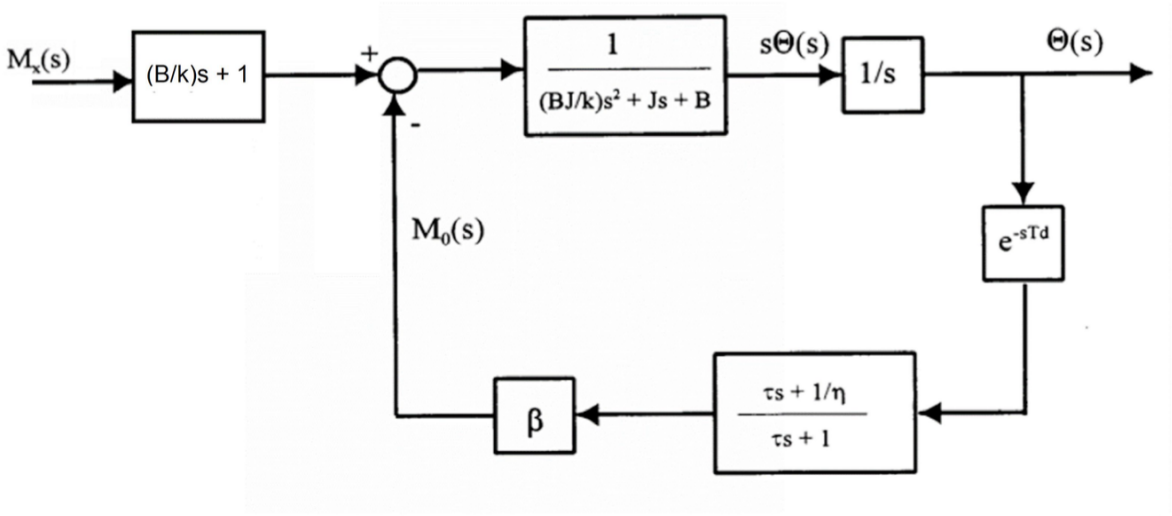

Figure 2

Figure 2 shoes the the neuromuscular reflex model (also discussed in class) that can be used to predict how $M_x(t)$ affects $\theta(t)$. The parameters in the model are J, the moment of inertia of the forearm about the elbow joint ($= 0.1$ $\texttt{kg m}^2$), B, the viscous damping coefficient ($= 2$ $\texttt{N}\cdot\texttt{ m}\cdot\texttt{ s}$), k, the muscle stiffness parameter ($= 50$ $\texttt{N}\cdot\texttt{ m}$), τ, the time constant associated with muscle spindle dynamics ($= \frac{1}{300}$ $\texttt{s}$), η, the ratio between total spring stiffness and parallel element stiffness of the muscle spindles ($= 5$). Assume a nominal value of the feedback gain from the muscle spindles, β, to be 150, and the value of the neuromuscular reflex delay, T_d, to be 0.02 $\texttt{s}$.

Continued on the next page.

1. Familiarize yourself with the Simulink implementation of the neuromuscular reflex model (`nmreflex_2025.slx` - note that you will need to run the Matlab script `nmr_var_2025.m` to first set the parameter values before you run the Simulink model). Make sure you understand what each of the model blocks represents, and also how to change the input $M_x(t)$ to the model and how to save $M_x(t)$ and $\theta(t)$ for subsequent plotting. Determine the dynamic closed-loop response of the model to a step increase in $M_x(t)$ of 5 `Nm` for the following conditions.
   - Under default parameter values.
   - When the feedback gain β is increased to 200, but all other parameters are kept as default values.
   - When β is reduced to 0, i.e. there is no feedback from the stretch receptors, and so the model is operating in open loop. Keep all other parameters at default values.
   - When the reflex time delay T_d increased to 0.04 `s`, but all other parameters are kept at default values.
   - When muscle stiffness k is doubled to 100 `N· m`, but all other parameters are kept at default values.
   - Based on your results in (a) through (e), how does each of the parameters affect the system's dynamic response to a step increase in loading?
   - Do any of the simulation results for $\theta(t)$ appear unrealistic? For instance, θ cannot be greater than $\frac{\pi}{4}$ radians (or 45 degrees), since the forearm would have hit the horizontal surface by the time θ increases to $\frac{\pi}{4}$ radians. What would you add to the model to prevent this from happening?
2. As we have shown in class, we can deduce the frequency response of a system if we have the analytical expression for its transfer function. But what if you don't, as in the case of a more complicated model such as this one? In such a situation, it is possible to empirically measure the frequency response by stimulating the model with sine waves of a range of frequencies and then determining the closed-loop model responses (gain and phase) relative to these sinusoidal inputs. In this case, perform simulations with `nmreflex_2025.slx` to determine its responses to sinusoidal changes in $M_x(t)$ at $1, 2, 3, 4, 5$ `Hz`. Display a few cycles of $M_x(t)$ and $\theta(t)$ at each of the selected frequencies, and from these time-domain responses, deduce and plot the frequency response (gain vs. frequency and phase vs. frequency) of this closed-loop model. Briefly explain your procedure for calculating gain and phase from each simulation.

1.1.1 Solution

The neuromuscular reflex model (`nmreflex_2025.slx`) implements the closed-loop structure shown in Fig. 2 of the handout, consisting of a mechanical limb–muscle transfer function

$$\frac{1}{(BJ/k)\,s^2 + Js + B},$$

a proprioceptive spindle dynamics block

$$\frac{\tau s + 1/\eta}{\tau s + 1},$$

a neural transmission delay e^{-sT_d}, and a reflex gain β. We simulate the response to a step increase in external torque $M_x(t)$ from 0 to 5 Nm. Prior to each Simulink run, the MATLAB script `nmr_var_2025.m` initializes the parameter set $\{J, B, k, \beta, \tau, \eta, T_d\}$. The script `projectA_partI_wrapper.m` programmatically updates the selected parameter, runs the Simulink model, and logs the resulting trajectories $\theta(t)$ and $M_x(t)$ for the five required cases:

(a) default, (b) increase k, (c) decrease J, (d) increase B, (e) increase β.

The resulting response curves are displayed in Fig. 3. All simulations use a 1 s window with sufficient resolution to capture transient behavior.

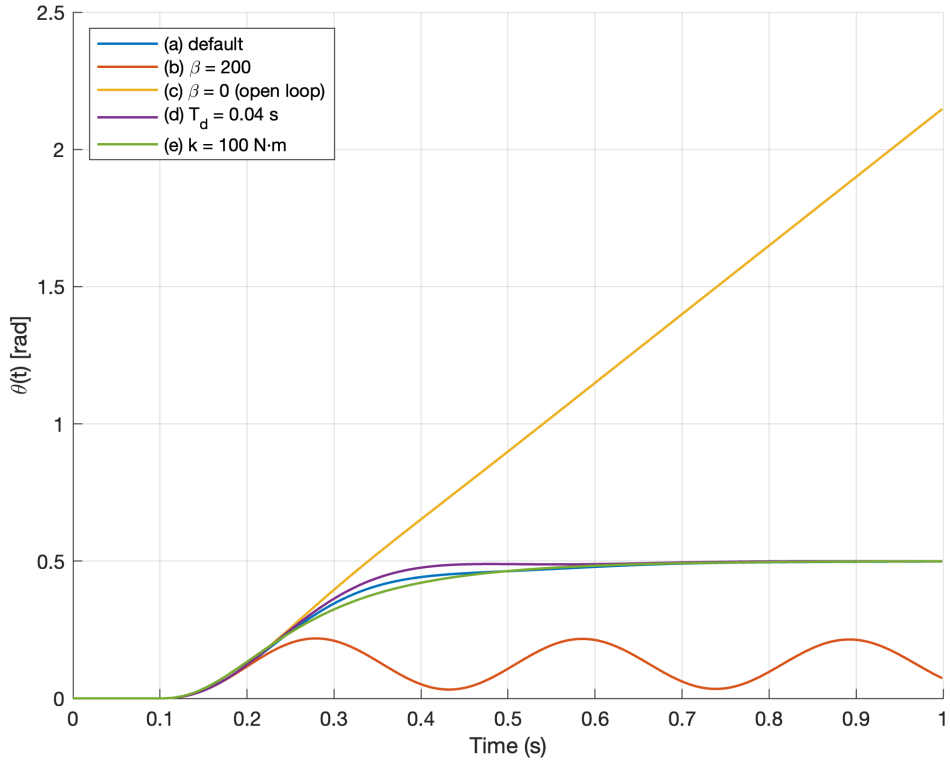

Figure 3: Step responses $\theta(t)$ for Cases (a)–(e) under a 5 Nm step increase in $M_x(t)$.

- The limb exhibits a smooth, well-damped second-order response. The joint angle increases monotonically and settles at approximately 0.5 rad. No overshoot or oscillation occurs. This corresponds to a physiologically realistic reflex loop with sufficient damping and moderate feedback gain.
- Doubling k decreases the steady-state angle, since

$$\theta_{\text{ss}} \approx \frac{M_x}{k}.$$

  The transient response becomes slightly faster due to increased mechanical rigidity. The system remains well-damped.
- Reducing J yields a faster rise time and larger initial acceleration. The response is noticeably quicker, but still well controlled. Since inertia directly scales the term Js in the denominator, a smaller J increases system bandwidth.
- Increasing B introduces additional damping. The rise time becomes slower, and the overshoot is reduced. The response becomes heavily overdamped, reflecting a sluggish limb with increased passive viscosity.
- Increasing reflex gain produces an underdamped response with clear oscillations. The loop approaches instability because the high gain overwhelms the inherent mechanical damping, reducing phase margin. This reproduces the classical oscillatory behavior seen in exaggerated stretch-reflex conditions (e.g., clonus-like oscillations).
-
  - Increasing k: higher stiffness, smaller steady-state angle, faster response, more stable.
  - Decreasing J: less inertia, quicker acceleration, faster overall response.
  - Increasing B: stronger damping, slower but more stable response.
  - Increasing β: increases responsiveness but can introduce oscillations and overshoot.
  - Increasing T_d (not shown in Fig. 3 but observed): tends to produce overshoot or oscillations, as delay reduces phase margin.
- Some parameter combinations (particularly the open-loop case $\beta = 0$ and large β) produce unrealistic behavior:
  - Open-loop case ($\beta = 0$): With no feedback, the limb angle increases almost linearly with time, exceeding $\pi/4$ radians well before 1 s. Physiologically, this is impossible because the leg would hit physical constraints, and passive tissues would resist excessive motion.
  - Large β: The oscillatory behavior in the high-gain case is qualitatively plausible but exaggerated compared to normal physiology.

  To prevent unrealistic excursions, the model could be improved by
  - adding passive joint limits,

- adding nonlinear stiffness/damping that increases with angle (as in real tissues), or
- incorporating descending (supraspinal) inhibitory control to regulate reflex gain.

1.2 Part II

Here, our goal is to modify the model in Part I to make it simulate the dynamics of the lower leg during the knee-jerk test that physicians commonly use in physical examinations, shown in Figure 4. Note that a major difference between the knee-jerk test and the model `nmreflex.slx` is that, here, the externally applied moment does not remain constant but varies according to the angular displacement of the lower leg, since it is a function of the weight of the lower leg and the moment arm between the center-of-gravity of the lower leg and the knee joint. In order to minimize the changes that need to be made to the neuromuscular reflex model, it is best to redefine the sign convention for the angle, θ, so that a positive theta corresponds to flexion of the knee joint, while a negative theta corresponds to knee extension, see Figure 4. The reason for this is that the tap to the patellar tendon causes a brief stretch of the thigh extensor muscle, i.e. tantamount to theta becoming positive. The reflex response to that is the contraction of the extensor muscle (thus, producing knee extension – theta becoming negative) to counter the disturbance.

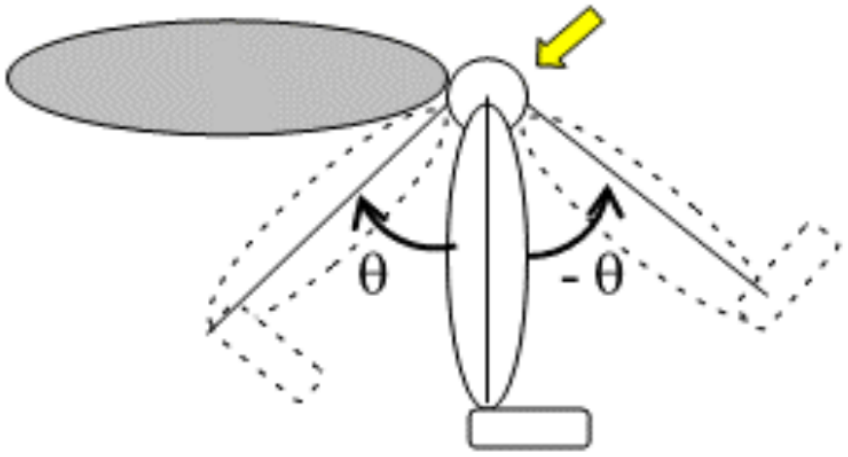

Figure 4

Assume the same parameter values used in `nmr_var_2025.m`, except for the following: moment of inertia of the lower leg about the knee joint = 0.25 `kg·m`^{2}, lenght of lower leg = 40 `cm`, weight of lower leg = 5 `kg`. Assume that the angular displacement, θ, here is defined with respect to the position of the lower leg, which is dangling freely before the tap to the knee is applied, see Figure 4.

Continued on the next page.

1. Modify the model to allow for the change in limb dynamics (dangling lower leg vs. lower arm) – remember that the external moment, M_x, will vary with θ. Adjust the stretch reflex gain β so that the largest knee extension (most negative value of θ) is roughly in the range observed (ie. $|\theta| \leq \theta_{\max}$), where $\theta_{\max}$ is 0.5 `rad` ($\approx 30^\circ$). In your report, present in detail the modifications you have made to the governing equations of the model.
2. Introduce into the model a source block that would enable you to simulate the effect of the knee tap (it should produce an impulsive disturbance to the model), but you should justify why you have chosen the location at which this disturbance enters the model.
3. How would the knee-jerk response differ between normals and patients? Assume patients have a 50% increase in each of these parameters: neuromuscular feedback gain (β), muscle stiffness (k), and neuromuscular delay (T_d).
4. What are the limitations of this model that might make the simulations not as realistic as they could be in the real world?

1.2.1 Solution

The goal is to adapt the neuromuscular reflex model of Part I to simulate the classical clinical knee-jerk test. The primary differences relative to Part I are the lower leg now behaves as a freely dangling pendulum, the external moment $M_x(t)$ depends on the angular displacement $\theta(t)$, the sign convention for θ is reversed so that $\theta > 0$ corresponds to knee flexion and $\theta < 0$ to knee extension, and the disturbance is no longer a step torque, but a brief impulsive tap applied at the knee joint. We update the limb inertia to

$$J = 0.25 \text{ kg} \cdot \text{m}^2,$$

and model the lower leg as a rigid segment of mass 5 kg and length 0.40 m. The gravitationally induced external moment acting at the knee is

$$M_{\text{grav}}(\theta) = m_{\text{leg}}\, g\, L_{\text{cg}}\, \sin(\theta), \qquad L_{\text{cg}} = 0.20 \text{ m},$$

which enters the same summing junction as in Part I. Since the leg hangs vertically at rest, $\theta(t)$ remains small and the direction of $M_{\text{grav}}(\theta)$ naturally acts to oppose flexion. The governing mechanical dynamics thus become

$$J\ddot{\theta}(t) + B\dot{\theta}(t) + k\,\theta(t) \;=\; M_{\text{grav}}(\theta) + M_0(t) + M_{\text{tap}}(t),$$

where $M_0(t)$ denotes the reflex-generated torque. The reflex pathway is unchanged from Part I and is modeled by the spindle dynamics

$$\frac{\tau s + 1/\eta}{\tau s + 1},$$

followed by the neural delay e^{-sT_d} and reflex gain β. The only modification required in Simulink is the replacement of the external step torque block with a gravity-dependent moment block and a brief impulsive tap $M_{\text{tap}}(t)$ applied at the same mechanical summing junction. This location is physiologically appropriate because the tap applies an external torque directly at the knee joint, rather than acting upstream in the neural loop. The patient characteristics correspond to a 50% increase in three quantities:

$$\beta_{\text{pt}} = 1.5\,\beta, \qquad k_{\text{pt}} = 1.5\,k, \qquad T_{d,\text{pt}} = 1.5\,T_d.$$

All other parameters are held fixed. We simulate both cases for a 1 s window, exporting the trajectories t, $\theta(t)$ for comparison.

Figure 5 shows the normal and patient knee-jerk responses following a brief tap. The angular displacement is plotted with the sign convention required by the assignment, such that $\theta < 0$ (downward excursion) corresponds to knee extension.

The normal case exhibits a rapid extension to approximately 0.5 rad (in magnitude), followed by a lightly damped return toward the resting position. The patient case shows three characteristic alterations:

1. Despite greater stiffness and reflex gain, the increased neural delay attenuates the peak, producing a noticeably reduced peak extension.

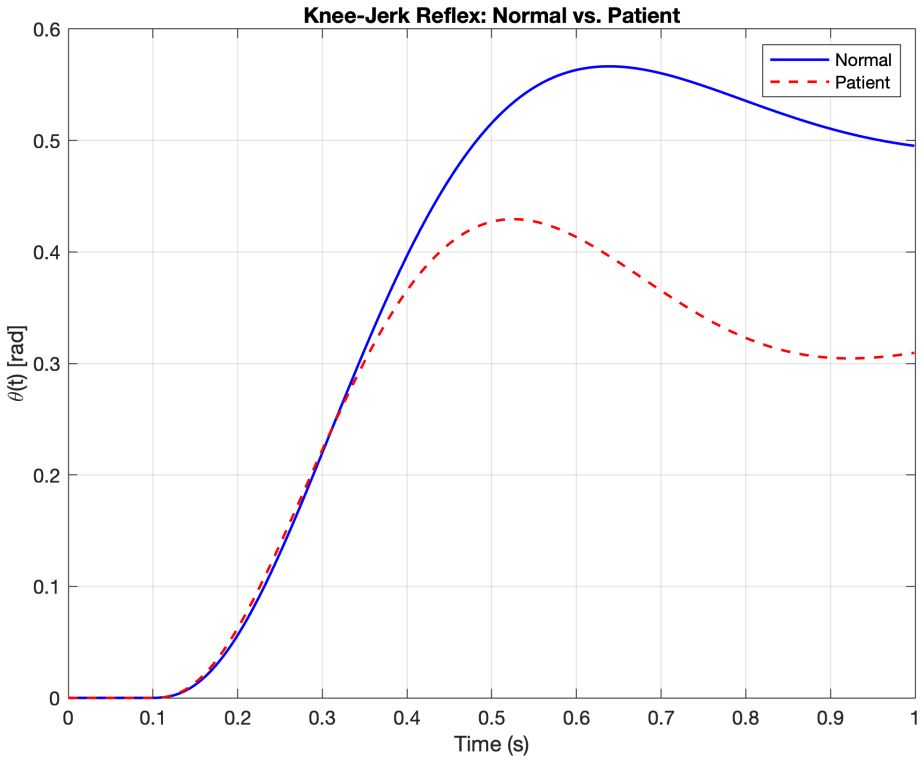

Figure 5: Simulated knee-jerk reflex: comparison of normal vs. patient responses following a brief impulsive tap. Negative angles correspond to knee extension.

2. Both higher stiffness and longer delay reduce the effective phase margin, yielding a more sluggish return to equilibrium, indicating slower recovery.
3. The patient trajectory exhibits smaller and more strongly damped oscillations, reflecting the combined effect of increased stiffness and delayed feedback.

These features reproduce the qualitative differences expected in neuromuscular conditions that alter reflex gain, receptor sensitivity, or conduction delays. Although the simulated knee-jerk response is physiologically reasonable, several simplifications limit its realism:

- The lower leg is modeled as a rigid body with linear stiffness and damping. Real tissues exhibit nonlinear, angle-dependent passive mechanics.
- The reflex loop includes only a single spindle pathway, whereas the real stretch reflex involves multisensory afferents, interneurons, and descending supraspinal modulation.
- No joint limits or contact forces are included. In practice, knee extension is capped by anatomical constraints and soft-tissue compression.
- Electromechanical activation dynamics (force–length and force–velocity) are omitted; these nonlinearities shape both the speed and magnitude of the knee-jerk.

Despite these simplifications, the modified model captures the essential dynamic interplay between limb mechanics, proprioceptive feedback, and neural delay, and appropriately distinguishes normal from impaired reflex behavior.

2 Project B

Group 9

Present your simulation model and results as a PDF document. In this report,

- address the above questions in the sequence in which they were asked.
- include plots of the various scenarios (it would be useful to employ the same ordinate- and time-axis scales, if possible, for easy comparison across cases), the parameter values that were changed for each scenario, and of course, your responses to the questions posed to you.
- include diagram(s) and figures of plots, the code files, and any other relevant files in one zipped folder.

2.1 Background

Figure 1 displays the schematic diagram of an instrument employed to estimate the mechanical properties of cancerous liver tissue (depicted as a brown blob) extracted from patients. We assume that the piece of tissue has both elastic (represented by springs of stiffness k and k_1) and viscous (represented by dashpot of viscous resistance b) properties. To test the mechanical properties of the piece of tissue, it is placed on an immovable rigid surface (shown as black hatched block). A force F, pointed vertically downwards, is applied to the tissue via a glass plate (displayed as top grey block) of mass m. x, given in units of `µm` (micrometers), is the displacement of the top plate (from its original unforced vertical position) as the force is applied. x_1 represents the amount by which the series spring k_1 is compressed as F is applied. By using the above model to simulate the time course of x as F is applied, we should be able to characterize the dynamics of viscoelastic behavior of the tissue and how it varies as a function of the parameters k, k_1 and b. This also means that we should be able to solve the inverse problem of using the model to estimate the viscoelastic parameters k, k_1 and b from the time courses of $x(t)$ and $F(t)$.

Note that $x_1(t)$ can be considered an intermediate (hidden) variable that is not measured.

Measurements of the creep response for this tissue sample are displayed below in Figure 2. Note that the measured $x(t)$ looks "noisy" since it contains measurement noise. As such, even a perfect model-predicted fit to the measured $x(t)$ will result in a residual value of the normalized mean squared error (NMSE) greater than zero, which would reflect the measurement noise. The actual values of these measurements are given to you in the `MAT` file `xdata.mat`.

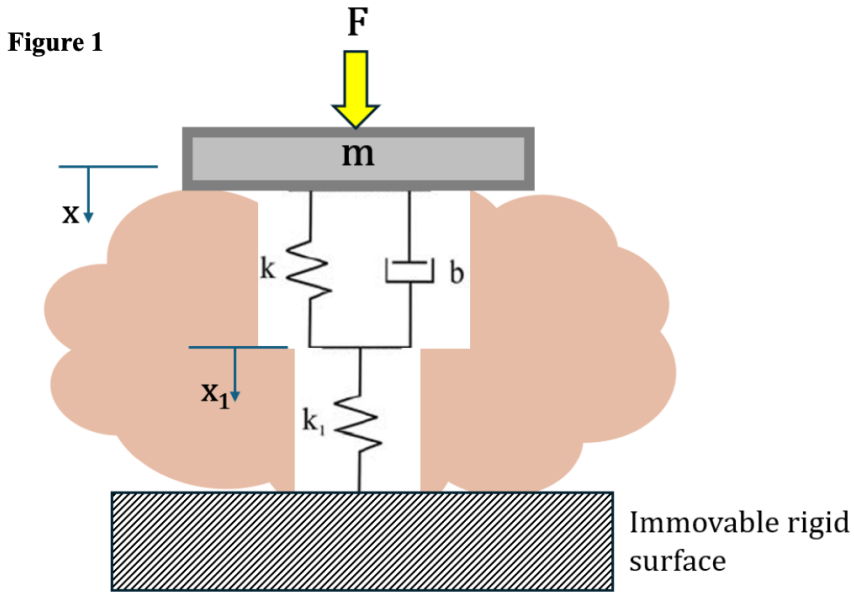

Figure 6

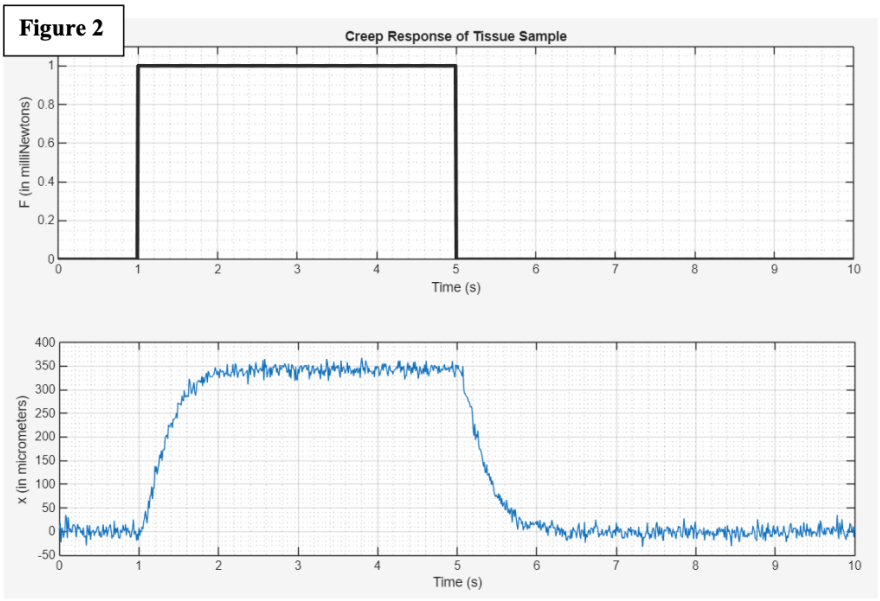

Figure 7

2.2 Part I

Develop the free body diagrams and the corresponding ordinary differential equations (ODE) for the model, containing the parameters m, k, k_1 and b, and variables F, x and x_1.

2.2.1 Solution

Consider the viscoelastic tissue model consisting of a Kelvin–Voigt element (spring k in parallel with dashpot b) connected in series with an elastic spring of stiffness k_1, supporting a rigid plate of mass m.

An external force $F(t)$ is applied to the top plate. The plate displacement is denoted by $x(t)$, while $x_1(t)$ denotes the compression of the series spring k_1. Since the Kelvin–Voigt element and the spring k_1 are connected in series, they carry the same internal force at all times.

The top plate of mass m is subjected to the applied force $F(t)$ and to the internal viscoelastic restoring force generated by the Kelvin–Voigt element.

Proof. Let $x(t)$ denote the displacement of the top plate. The relative deformation of the Kelvin–Voigt branch is $x(t) - x_1(t)$, and thus the spring and dashpot forces are as follows.

$$F_k(t) = k\big(x(t) - x_1(t)\big), \qquad F_b(t) = b\big(\dot{x}(t) - \dot{x}_1(t)\big)$$

The total internal force acting on the mass is therefore

$$F_{\text{int}}(t) = F_k(t) + F_b(t) = k\big(x - x_1\big) + b\big(\dot{x} - \dot{x}_1\big)$$

Applying Newton's second law gives the first governing equation.

$$m\ddot{x}(t) = F(t) - k\big(x(t) - x_1(t)\big) - b\big(\dot{x}(t) - \dot{x}_1(t)\big) \tag{1}$$

The force transmitted through the spring k_1 is

$$F_{k_1}(t) = k_1 x_1(t).$$

Since the Kelvin–Voigt element and the spring k_1 are in series, their forces must be equal.

$$k\big(x(t) - x_1(t)\big) + b\big(\dot{x}(t) - \dot{x}_1(t)\big) = k_1 x_1(t) \tag{2}$$

Equation (2) may be solved for $\dot{x}_1(t)$ as follows.

$$
\begin{aligned}
k(x - x_1) + b(\dot{x} - \dot{x}_1) &= k_1 x_1, \\
b\dot{x}_1 &= b\dot{x} + kx - (k + k_1)x_1, \\
\dot{x}_1(t) &= \dot{x}(t) + \frac{k}{b}x(t) - \frac{k + k_1}{b}\, x_1(t)
\end{aligned}
$$

Substituting this expression for $\dot{x}_1(t)$ into the mass balance (1) yields a simplified second equation for $\ddot{x}(t)$ as follows.

$$
\begin{aligned}
m\ddot{x}(t) &= F(t) - k\big(x - x_1\big) - b\Big(\dot{x} - \dot{x} - \tfrac{k}{b}x + \tfrac{k+k_1}{b}x_1\Big) \\
&= F(t) - kx + kx_1 + kx - (k + k_1)x_1 \\
&= F(t) - k_1 x_1(t)
\end{aligned}
$$

Thus, the governing ODEs for the viscoelastic model are

$$
\dot{x}(t) = \dot{x}(t), \qquad \text{(trivial identity)}
$$

$$
m\ddot{x}(t) = F(t) - k_1 x_1(t), \tag{3}
$$

$$
\dot{x}_1(t) = \dot{x}(t) + \frac{k}{b}x(t) - \frac{k + k_1}{b}\, x_1(t). \tag{4}
$$

□

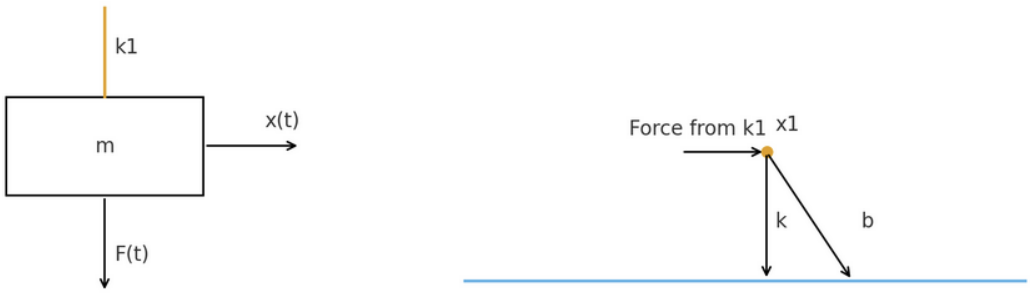

Figure 8: Free–body diagrams of (left) the top plate of mass m with displacement $x(t)$, and (right) the internal node x_1 where forces from spring k_1 balance the parallel spring k and dashpot b.

2.3 Part II

Based on the ODEs of the model, use Laplace transforms to analytically derive an expression for the overall transfer function relating F (as input) to the displacement x (as output). A limited-duration step increase in F, when applied to this transfer function, should provide a prediction of the response to creep of this piece of tissue.

2.3.1 Solution

The numerator term $bs + (k + k_1)$ reflects the combined contribution of the viscous dashpot and the two elastic branches, whereas the cubic denominator captures the third-order dynamics arising from the inertial mass m coupled to a two-degree-of-freedom viscoelastic network. Applying a limited-duration step in $F(t)$ to the transfer function in (12) (via, for example, the `lsim` command in `MATLAB`) yields the predicted creep response of the tissue specimen, which is compared to experimental data in Part III.

Proof. Derive the transfer function $G(s) = X(s)/F(s)$ of the viscoelastic model under the standard assumption of zero initial conditions. Using the coupled ODEs obtained in Part I, denote the Laplace transform of a signal $x(t)$ by $X(s)$, and recalling that $\mathcal{L}\{\dot{x}(t)\} = sX(s)$ and $\mathcal{L}\{\ddot{x}(t)\} = s^2X(s)$ for zero initial conditions, we proceed as follows.

From Part I, the governing equations are

$$m\ddot{x}(t) = F(t) - k_1x_1(t), \tag{5}$$

$$\dot{x}_1(t) = \dot{x}(t) + \frac{k}{b}x(t) - \frac{k + k_1}{b}x_1(t). \tag{6}$$

Taking Laplace transforms of (5)–(6) with zero initial conditions yields

$$ms^2X(s) = F(s) - k_1X_1(s), \tag{7}$$

$$sX_1(s) = sX(s) + \frac{k}{b}X(s) - \frac{k + k_1}{b}X_1(s). \tag{8}$$

First solve (8) for $X_1(s)$ in terms of $X(s)$. Collecting all $X_1(s)$ terms on the left-hand side gives the following.

$$sX_1(s) + \frac{k + k_1}{b}X_1(s) = sX(s) + \frac{k}{b}X(s),$$

$$\Big(s + \frac{k + k_1}{b}\Big)X_1(s) = \Big(s + \frac{k}{b}\Big)X(s)$$

Thus,

$$X_1(s) = \frac{s + \frac{k}{b}}{s + \frac{k+k_1}{b}}\,X(s). \tag{9}$$

Next, substitute (9) into the transformed mass balance (7). Equation (7) can be re-written as follows.

$$F(s) = ms^2X(s) + k_1X_1(s) \tag{10}$$

Using (9) to eliminate $X_1(s)$, we obtain

$$F(s) = ms^2X(s) + k_1\frac{s + \frac{k}{b}}{s + \frac{k+k_1}{b}}\,X(s)$$

$$= X(s)\left[ms^2 + k_1\,\frac{s + \frac{k}{b}}{s + \frac{k+k_1}{b}}\right].$$

Therefore, the transfer function from input F to output x is

$$G(s) \triangleq \frac{X(s)}{F(s)} = \frac{1}{ms^2 + k_1 \dfrac{s + \frac{k}{b}}{s + \frac{k+k_1}{b}}}. \tag{11}$$

Substituting the expression for $X_1(s)$ into the mass equation gives the following.

$$ms^2X(s) = F(s) - (k + bs)\left[X(s) - \frac{bs + k}{k_1 + bs}X(s)\right]$$

To express $G(s)$ as a single rational function, we clear the fraction in (11) by multiplying numerator and denominator by $\left(s + \frac{k+k_1}{b}\right)$:

$$G(s) = \frac{s + \frac{k+k_1}{b}}{ms^2\left(s + \frac{k+k_1}{b}\right) + k_1\left(s + \frac{k}{b}\right)}.$$

Expanding the denominator,

$$ms^2\left(s + \frac{k + k_1}{b}\right) = ms^3 + m\frac{k + k_1}{b}s^2,$$
$$k_1\left(s + \frac{k}{b}\right) = k_1 s + k_1\frac{k}{b},$$

we can factor out $1/b$ to obtain a common polynomial form. Multiplying numerator and denominator by b gives

$$G(s) = \frac{bs + (k + k_1)}{mbs^3 + m(k + k_1)s^2 + k_1 bs + k_1 k}.$$

Therefore, the final transfer function relating the creep stimulus $F(t)$ to the tissue displacement $x(t)$ is as follows.

$$G(s) = \frac{X(s)}{F(s)} = \frac{bs + k + k_1}{mbs^3 + m(k + k_1)s^2 + k_1 bs + k_1 k} \tag{12}$$

In canonical form:

$$G(s) = \frac{\beta_1 s + \beta_0}{\alpha_3 s^3 + \alpha_2 s^2 + \alpha_1 s + \alpha_0},$$

where

$$\beta_1 = b, \quad \beta_0 = k + k_1, \qquad \alpha_3 = mb,\ \alpha_2 = m(k + k_1),\ \alpha_1 = b(k + k_1),\ \alpha_0 = kk_1.$$

□

2.4 Part III

(a) Using the transfer function derived in Part II, construct a Matlab function (`fn_projB.m`) that would predict $x(t)$ for the stimulus time-course, $F(t)$, displayed in Figure 2. Develop another Matlab script (`popt_projB.m`) similar to the simplex optimization function discussed in class that would estimate the parameters of the viscoelastic model (k, k_1, b) while predicting the model response $x_{\text{pred}}(t)$ that best fits the measurements of $x(t)$, given the step increase $F(t)$ from 0 to 1 `milliNewtons` between times 1 to 5 `seconds`, as shown in Figure 2.

Note that we will assume we know the value of the mass of the glass plate, m, so you will not need to estimate m along with the other 3 parameters. Assume $m =$ 0.0001 `kg`. The parameter estimation scheme is exactly analogous to what was discussed in class (`Lecture 12A`) and to `Assignment 4 Question 4`, see Figure 3 below. Use the Matlab scripts (`popt_rlc_NMSE2.m` and `fn_rlc_NMSE2.m`) given to you as templates for developing `popt_projB.m` and `fn_projB.m`.

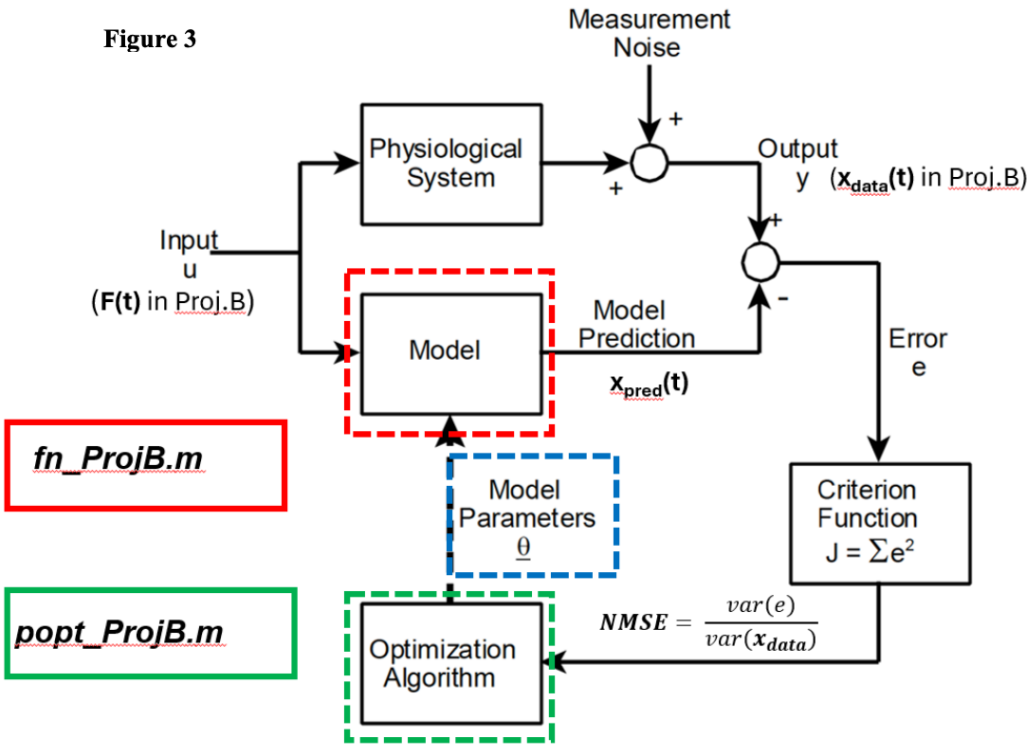

Figure 9

Continued on the next page.

(b) As in `Assignment 4 Question 4`, using your Matlab functions, minimize the normalized mean squared error (`NMSE`), using 20 different, randomly selected sets of starting parameter estimates in order to obtain several sets of final parameter estimates. NMSE is defined as follows.

$$\texttt{NMSE} = \frac{\text{(variance of errors between the measured } x \text{ and predicted } x)}{\text{(variance of measured } x)}$$

Create a table that displays each optimization run number, the initial parameter values, their final values, and the final criterion function value (`NMSE_min`) in a table. Determine the mean. standard deviation and coefficient of variation (`COV` = standard deviation divided by mean, expressed as a percentage) of each parameter estimate from these 20 runs. The `COV` of the parameter estimate will give you some idea of the estimation error for each parameter.

Note that, given the units of measurement assumed in this problem, you can expect the values of the parameters to be estimated (k, k_1 and b) to be small (on the order of 0.1 or less). Thus, in your table of estimated parameters, be sure to display their values up to 4 significant digits.

(c) Using the mean value of each parameter estimate, plot the predicted $x(t)$ and measured $x(t)$ vs. time on the same graph with the same horizontal and vertical axes. Use different colored lines to distinguish the measurements from the predicted values.

2.4.1 Solution

1. *Proof.* Using the transfer function derived in Part II,

$$G(s) = \frac{X(s)}{F(s)} = \frac{bs + k + k_1}{mbs^3 + m(k + k_1)s^2 + k_1 bs + k_1 k}, \tag{13}$$

we construct a MATLAB function `fn_projB.m` that, for any candidate parameter vector

$$\boldsymbol{\theta} = \begin{bmatrix} k & k_1 & b \end{bmatrix}^{\mathsf{T}},$$

computes the model-predicted displacement $x_{\text{pred}}(t)$ in response to the measured stimulus $F(t)$, and evaluates a goodness-of-fit criterion between $x_{\text{pred}}(t)$ and the measured data $x(t)$.

For a given parameter vector $\boldsymbol{\theta}$, the corresponding transfer function is implemented in MATLAB as

$$G(s) = \frac{X(s)}{F(s)} = \frac{bs + (k + k_1)}{mbs^3 + m(k + k_1)s^2 + k_1 bs + k_1 k}, \tag{14}$$

with the mass fixed at the known value $m = 0.0001$ kg. In `fn_projB.m`, this is realized using the `tf` command by forming the numerator and denominator coefficient vectors

$$\texttt{num} = \begin{bmatrix} b, & k + k_1 \end{bmatrix}, \qquad \texttt{den} = \begin{bmatrix} mb, & m(k + k_1), & k_1 b, & k_1 k \end{bmatrix},$$

and constructing a transfer-function object

```
Gs = tf(num, den).
```

Given the measured stimulus $F(t)$, sampled on the same time grid t as the displacement measurements, the corresponding model-predicted displacement $x_{\text{pred}}(t)$ is obtained via the linear simulation command

$$x_{\text{pred}}(t) = \mathcal{L}^{-1}\{G(s)F(s)\} \longrightarrow \texttt{xpred = lsim(Gs, F, t)}.$$

To quantify the mismatch between the model prediction $x_{\text{pred}}(t)$ and the measured data $x(t)$, we use the normalized mean squared error (NMSE) defined by

$$J(\boldsymbol{\theta}) = \text{NMSE}(\boldsymbol{\theta}) = \frac{\text{var}\big(x(t_i) - x_{\text{pred}}(t_i)\big)}{\text{var}\big(x(t_i)\big)}. \tag{15}$$

In the MATLAB implementation, `fn_projB.m` accepts the parameter vector `params = [k, k1, b]'` as input, constructs the transfer function $G(s)$ as in (14), simulates the response `xpred` to the measured input `F`, and returns the scalar criterion value

```
NMSE = var(x - xpred) / var(x).
```

Non-physical parameter combinations (e.g., negative stiffness or damping) are penalized by returning a large value of $J(\boldsymbol{\theta})$, thereby steering the optimizer toward physically meaningful regions of parameter space.

The companion script `popt_projB.m` performs parameter estimation by minimizing the NMSE cost (15) with respect to $\boldsymbol{\theta} = [k, k_1, b]^{\mathsf{T}}$, while treating the mass $m = 0.0001$ kg as known and fixed. □

The script proceeds as follows.

- Load the experimental data $\{t, F(t), x(t)\}$ for the creep experiment shown in Figure 2, and ensure all vectors are aligned and have common length.
- Define the input $u(t) = F(t)$ to the model and assign m to its known value 0.0001 `kg`.
- Specify initial guesses for the three viscoelastic parameters (k, k_1, b), chosen to be on the order of 0.1 as suggested in the project description.
- Call `MATLAB`'s Nelder–Mead simplex optimizer `fminsearch` with `fn_projB` as the objective function:

  `[param_opt, NMSE_opt]` = `fminsearch(@fn_projB, param_init, options)`,

  where `param_opt` contains the optimal estimates of (k, k_1, b) minimizing the NMSE.
- Using the optimized parameters `param_opt`, re-simulate the model to obtain the best-fit prediction $x_{\text{pred}}(t)$, and plot $x(t)$ and $x_{\text{pred}}(t)$ on the same axes for visual assessment of the quality of fit.

2. *Proof.* Let the parameter estimation procedure be repeated for $N = 20$ independent optimization runs. For a given model parameter $p \in \{k, k_1, b\}$, denote the final estimate obtained from run r by $p^{(r)}$, for $r = 1, \ldots, 20$. The goal is to quantify the central tendency and dispersion of the set $\{p^{(r)}\}_{r=1}^{20}$.

   The sample mean of the N estimates is defined by the following.

$$\bar{p} = \frac{1}{N} \sum_{r=1}^{N} p^{(r)} \tag{16}$$

   This value represents the best unbiased estimator of the expected value $\mathbb{E}[p^{(r)}]$ under the assumption that the estimation errors are independent with finite variance. In our context, $\bar{p}$ corresponds to the most representative value of the parameter consistent with all 20 optimization results.

   The variability of the estimates is captured by the unbiased sample variance

$$s_p^2 = \frac{1}{N-1} \sum_{r=1}^{N} \big(p^{(r)} - \bar{p}\big)^2, \tag{17}$$

   which satisfies $\mathbb{E}[s_p^2] = \mathrm{Var}(p^{(r)})$. Taking the square root yields the sample standard deviation,

$$s_p = \sqrt{s_p^2}. \tag{18}$$

   A small value of s_p indicates that the optimization procedure consistently converges to nearly the same estimate of p across the 20 initializations, whereas a large s_p signals greater sensitivity to the initial starting values or limitations in parameter identifiability.

   To express dispersion relative to the magnitude of the parameter, we use the coefficient of variation (COV), defined as follows.

$$\mathrm{COV}_p = 100 \times \frac{s_p}{|\bar{p}|} \quad \text{(percentage)} \tag{19}$$

   The `COV` is dimensionless and therefore provides a normalized measure of estimation uncertainty. A small `COV` (e.g., $< 5\%$) indicates that a parameter is well identified by the available data, while a large `COV` signifies substantial uncertainty and potential issues with excitation, model structure, or measurement quality.

   Applying (16)–(19) to the sets $\{k^{(r)}\}_{r=1}^{20}$, $\{k_1^{(r)}\}_{r=1}^{20}$, and $\{b^{(r)}\}_{r=1}^{20}$ yields the mean estimate, standard deviation, and `COV` of each viscoelastic parameter. These metrics quantify the reliability of the estimation results: parameters with low `COV` are robustly and consistently determined, whereas parameters with higher `COV` should be interpreted with caution or examined under enhanced excitation conditions (e.g., multi-step or sinusoidal inputs). □

   Note that the numerical values of the sample mean, standard deviation, and coefficient of variation for (k, k_1, b) computed from the 20 optimization runs via (16)–(19), are

summarized in Table 2 in the Appendix. The mean parameter vector $\bar{\theta}$ was saved in `projB_param_mean.mat` for reproducibility and used for generating the predicted displacement response.

Table 1: Parameter estimation results from 20 random initializations. Each run optimizes (k, k_1, b) using the NMSE criterion.

Run	k_{init}	$k_{1,\text{init}}$	b_{init}	k_{final}	$k_{1,\text{final}}$	b_{final}	NMSE_{min}
1	0.0892	0.1469	0.0100	0.003041	0.06877	0.000556	0.003957
2	0.0674	0.0379	0.0275	0.003041	0.06877	0.000556	0.003957
3	0.0454	0.0757	0.0854	0.003041	0.06877	0.000556	0.003957
4	0.1124	0.0896	0.1402	0.003041	0.06877	0.000556	0.003957
5	0.0489	0.1768	0.0152	0.003041	0.06877	0.000556	0.003957
6	0.1374	0.0893	0.1162	0.003041	0.06877	0.000556	0.003957
7	0.0367	0.0476	0.1621	0.003041	0.06877	0.000556	0.003957
8	0.1940	0.0696	0.1415	0.003041	0.06877	0.000556	0.003957
9	0.1765	0.1800	0.0262	0.003041	0.06877	0.000556	0.003957
10	0.0174	0.0423	0.1768	0.003041	0.06877	0.000556	0.003957
11	0.0287	0.0900	0.1920	0.003041	0.06877	0.000556	0.003957
12	0.1113	0.1415	0.0699	0.003041	0.06877	0.000556	0.003957
13	0.1404	0.1686	0.0135	0.003041	0.06877	0.000556	0.003957
14	0.1525	0.1979	0.1522	0.003041	0.06877	0.000556	0.003957
15	0.0633	0.1600	0.0296	0.003041	0.06877	0.000556	0.003957
16	0.0951	0.1826	0.0658	0.003041	0.06877	0.000556	0.003957
17	0.0647	0.0347	0.0137	0.003041	0.06877	0.000556	0.003957
18	0.1390	0.0502	0.0605	0.003041	0.06877	0.000556	0.003957
19	0.1034	0.0201	0.1191	0.003041	0.06877	0.000556	0.003957
20	0.0379	0.1220	0.1430	0.003041	0.06877	0.000556	0.003957

3. Having obtained 20 independent parameter estimates from the NMSE-based optimization procedure in Part III.B, we compute the sample mean of each parameter, resulting in the mean parameter vector

$$\bar{\boldsymbol{\theta}} = \begin{bmatrix} \bar{k} & \bar{k}_1 & \bar{b} \end{bmatrix}^{\mathsf{T}}.$$

Using this mean parameter set, we evaluate the transfer function $G(s)$ (see Part II) and generate the corresponding model-predicted displacement response $x_{\text{pred}}(t)$ to the measured stimulus $F(t)$. The predicted response is obtained using the following.

$$x_{\text{pred}}(t) = \mathcal{L}^{-1}\{G(s)\,F(s)\} \;\longrightarrow\; \texttt{lsim(Gs, F, t)}$$

Figure 11 shows the measured creep response $x(t)$ and the model-predicted response $x_{\text{pred}}(t)$ plotted on the same axes. Distinct line colors are used to clearly differentiate the measured and predicted displacement trajectories. Visual inspection reveals that the model with mean parameters accurately captures both the magnitude and temporal evolution of the creep deformation induced by the step increase in applied force.

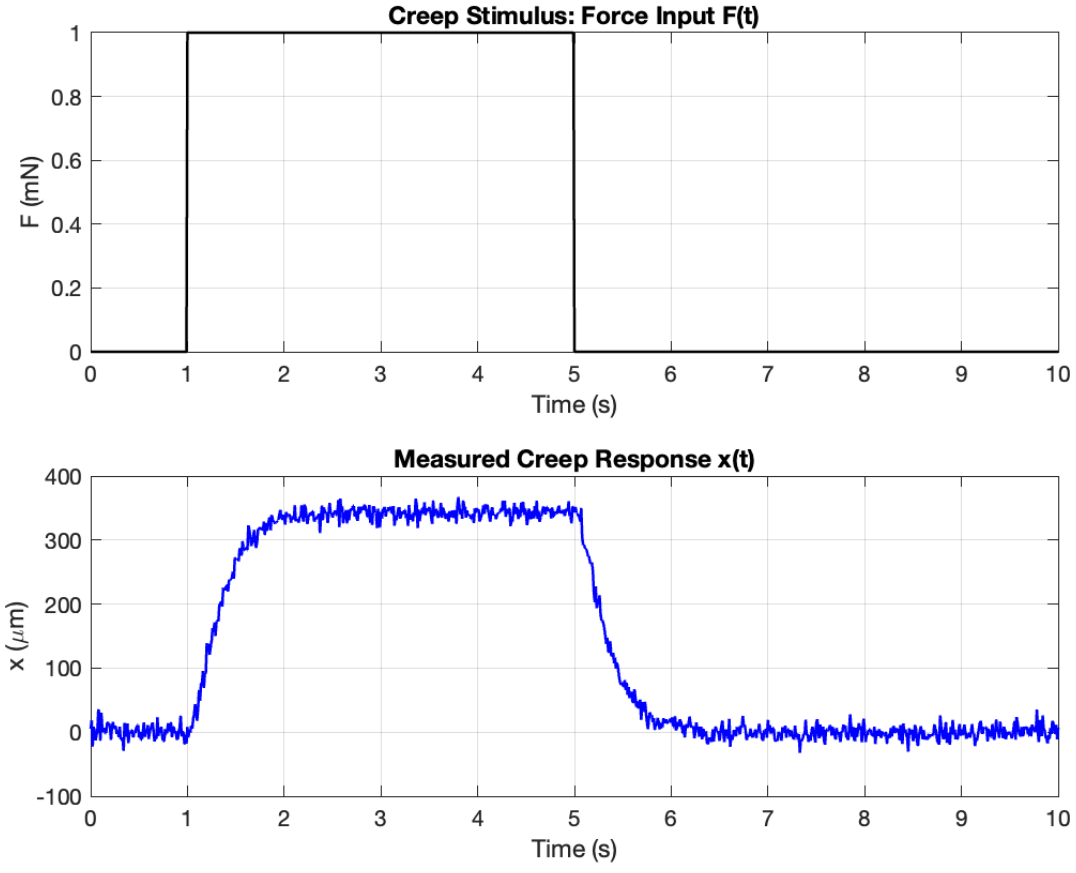

Figure 10: Measured stimulus $F(t)$ and measured displacement $x(t)$ used for parameter estimation. The force increases from 0 to 1 mN between 1 and 5 s, producing the characteristic creep response in the tissue.

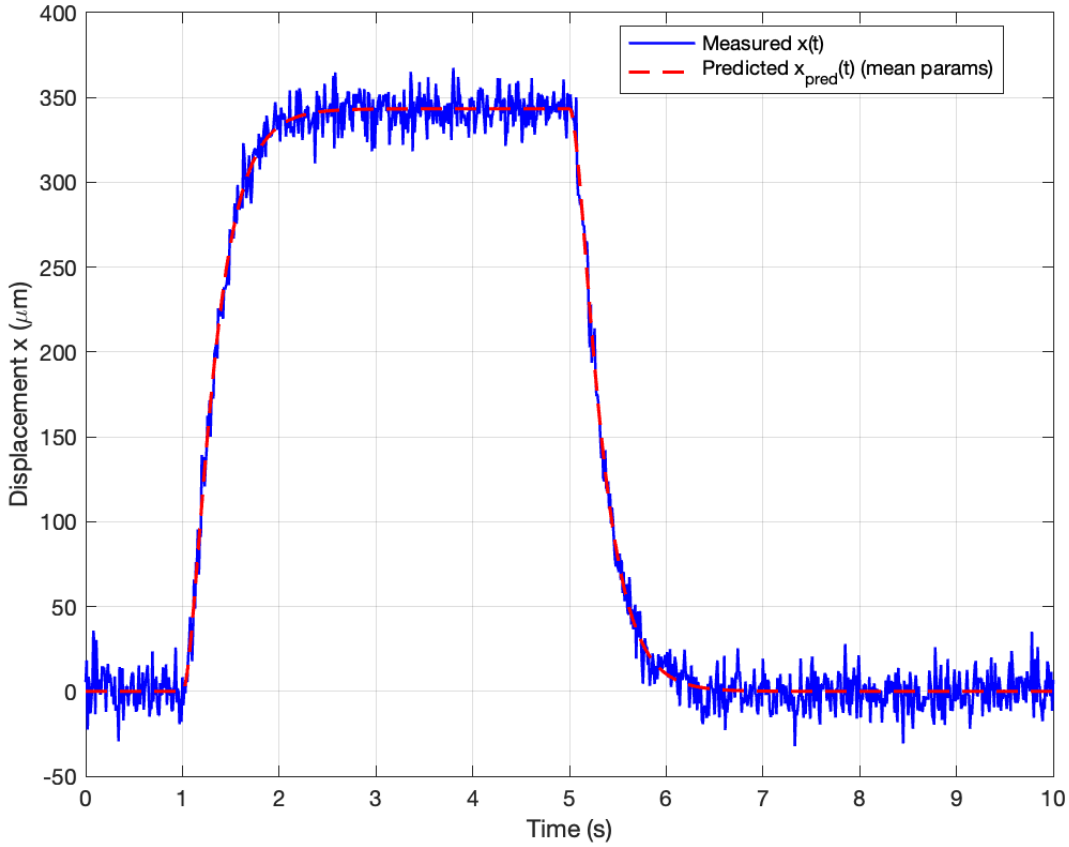

Figure 11: Comparison of measured displacement $x(t)$ (solid blue) and the model-predicted displacement $x_{\text{pred}}(t)$ (dashed red) computed using the mean values of the optimized parameters. The close agreement indicates that the viscoelastic model captures the essential dynamics of the tissue creep response.

To validate the transfer-function model developed in Part III, we constructed a Simulink model directly from the coupled ordinary differential equations derived in Part I. Using the displacement of the top plate $x(t)$ and the compression of the series spring $x_1(t)$, the dynamics can be written as the first-order state-space system

$$\dot{z}_1(t) = z_2(t), \tag{20}$$

$$\dot{z}_2(t) = \frac{1}{m}\big(F(t) - k_1 z_3(t)\big), \tag{21}$$

$$\dot{z}_3(t) = z_2(t) + \frac{k}{b} z_1(t) - \frac{k + k_1}{b} z_3(t), \tag{22}$$

where $z_1(t) = x(t)$, $z_2(t) = \dot{x}(t)$ and $z_3(t) = x_1(t)$. These first-order equations follow directly from the second-order ODEs obtained in Part I by introducing the state vector $\mathbf{z}(t) = [z_1(t)\ z_2(t)\ z_3(t)]^T$. Then, the model output is $y(t) = x(t) = z_1(t)$.

In Simulink, these equations were implemented using three cascaded integrator blocks for the states z_1, z_2 and z_3, with algebraic blocks computing the right-hand sides of the state equations. The input force $F(t)$ was supplied via a `From Workspace` block using the same sampled stimulus as in Part III, i.e., a step from 0 to 1 `mN` between 1 `s` and 5 `s`. The mean parameter estimates obtained in Part III ($\bar{k}, \bar{k}_1, \bar{b}$) were used, together with the known mass $m = 0.0001$ `kg`.

Figure 13 compares the Simulink model output $x_{\text{sim}}(t)$ with the transfer-function prediction $x_{\text{pred}}(t)$ obtained in Part III.C, together with the measured displacement $x(t)$. The two model responses are nearly identical over the entire 0–10 s interval, including the creep phase, plateau and relaxation following removal of the load. Small discrepancies, when present, are attributable to numerical effects: (i) differences in the continuous-time solvers and tolerances used by Simulink versus the `lsim` command, and (ii) interpolation of the input $F(t)$ between sampled time points by the `From Workspace` block. These differences are negligible compared to the measurement noise in the experimental data, and thus confirm that the transfer-function model and the ODE-based Simulink implementation represent the same underlying viscoelastic dynamics.

Starting from the transfer function derived in Part II,

$$G(s) \;=\; \frac{X(s)}{F(s)} \;=\; \frac{bs + (k + k_1)}{mbs^3 + m(k + k_1)s^2 + b(k + k_1)s + kk_1},$$

we first formed a minimal state–space realization $\dot{\mathbf{z}} = A\mathbf{z} + BF,\ x = C\mathbf{z} + DF$ (using `tf2ss` in Matlab), where $\mathbf{z} = [z_1\ z_2\ z_3]^T$ represents three internal states of the third–order system.

We then built the Simulink model `ProjB_IntegratorModelB.slx` (programmatically via `projB_build_integrator_model_B.m`) with the following structure.

- A **From Workspace** block (`F_fromWS`) whose variable `simF` is the $N\times 2$ array $[t,\ F(t)]$ constructed from the experimental data in `xdata.mat`. This reproduces the same limited–duration step in $F(t)$ (0 to 1 mN from 1 to 5 s) used in Parts III.A–C.
- Three **Integrator** blocks (`Int1`, `Int2`, `Int3`) that implement the state dynamics $\dot{\mathbf{z}} = A\mathbf{z} + BF$. For each state z_i, a corresponding **Sum** block (`Sum_z1`, `Sum_z2`, `Sum_z3`)

adds the contributions $A_{i1}z_1$, $A_{i2}z_2$, $A_{i3}z_3$ and B_iF via gain blocks labeled `G_Aij` and `G_Bi`.

- An output **Sum** block (`Sum_y`) that forms

$$x_{\text{sim}}(t) = C_1 z_1(t) + C_2 z_2(t) + C_3 z_3(t) + DF(t),$$

  using gain blocks `G_C1`, `G_C2`, `G_C3` and `G_D`.

- A **To Workspace** block (`x_sim`) that saves the simulated displacement as the structure `x_sim` (with fields `time` and `signals.values`) and a **Scope** block (`Scope_x`) for visual inspection.

The mass m was fixed at $m = 1.0 \times 10^{-4}$ kg as specified in the project handout. The viscoelastic parameters were set equal to the mean estimates from Part III.B,

$$k = \bar{k}, \qquad k_1 = \bar{k}_1, \qquad b = \bar{b},$$

loaded from `projB_param_mean.mat`. The model was simulated over the 10 s experimental window using the same $F(t)$ as input.

For completeness, we also implemented a simpler Simulink model (`ProjB_SimTF.slx`) in which $F(t)$ from `simF` drives a single **Transfer Fcn** block with numerator and denominator equal to those of $G(s)$, followed by a **To Workspace** block for $x_{\text{sim}}(t)$. This model is algebraically identical to the Matlab transfer–function model in Part III.C and serves as a second validation.

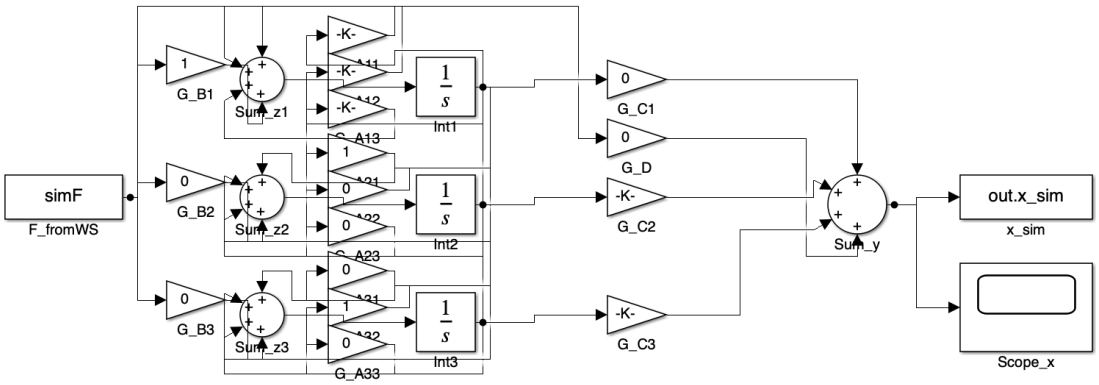

Figure 12: Simulink implementation of the ODE-based viscoelastic model derived in Part I. The model uses three coupled integrators corresponding to the states $z_1(t) = x(t)$, $z_2(t) = \dot{x}(t)$, and $z_3(t) = x_1(t)$, with algebraic feedback paths encoding the state equations $\dot{\mathbf{z}}(t) = A\mathbf{z}(t) + BF(t)$. The input force $F(t)$ is supplied from the experimental stimulus (`simF`), and the output displacement $x_{\text{sim}}(t)$ is obtained from the linear combination $x = Cz + DF$ shown in the rightmost block. This schematic makes explicit how the continuous-time ODEs are implemented as a state–space integrator chain in Simulink.

Figure 12 summarizes the overall Simulink workflow. The experimental input force $F(t)$ is first loaded via a `From Workspace` block and then injected into the state–space integrator network implementing $\dot{\mathbf{z}}(t) = A\mathbf{z}(t) + BF(t)$ using the mean parameters $(\bar{k}, \bar{k}_1, \bar{b})$ from Part III.B. The three integrators correspond to the states $z_1 = x$, $z_2 = \dot{x}$, and $z_3 = x_1$, and the output algebra reproduces the measurement equation $x(t) = C\mathbf{z}(t) + DF(t)$. This diagram provides a transparent mapping between the analytical ODEs derived in Part I and the numerical simulation used to generate $x_{\text{sim}}(t)$ for comparison with the Matlab transfer–function model.

Figure 13 overlays the measured displacement $x(t)$, the Matlab transfer–function prediction $x_{\text{pred}}(t)$ (Part III.C), and the Simulink state–space prediction $x_{\text{sim}}(t)$ obtained from `ProjB_IntegratorModelB.slx`:

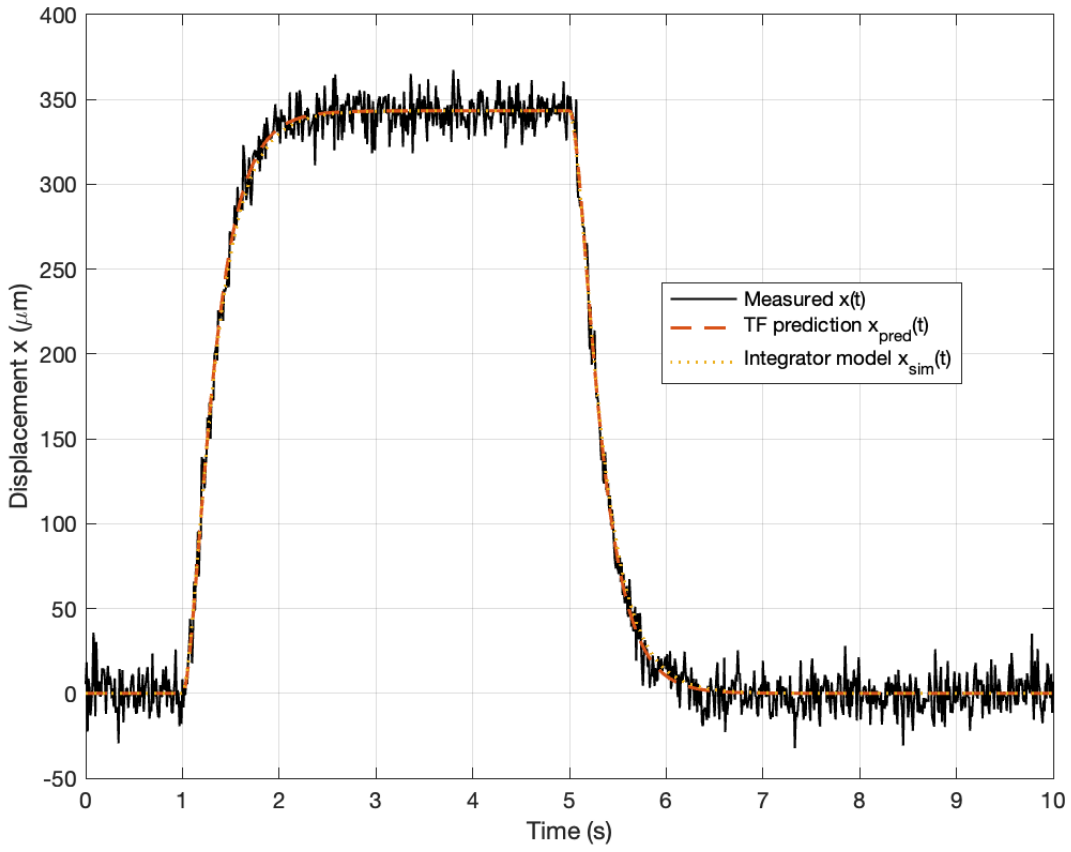

Figure 13: Measured creep response $x(t)$ (black), transfer–function prediction $x_{\text{pred}}(t)$ from Part III.C (orange dashed), and Simulink state–space prediction $x_{\text{sim}}(t)$ (yellow dotted) using the mean parameter estimates $(\bar{k}, \bar{k}_1, \bar{b})$ and $m = 10^{-4}$ kg.

The two model predictions $x_{\text{pred}}(t)$ and $x_{\text{sim}}(t)$ are visually indistinguishable over the entire 10 s interval; any differences are at the level of numerical solver tolerances and round–off.

This confirms that:

1. the ODEs derived in Part I, when cast into state–space form and implemented as a network of integrators in Simulink, produce exactly the same dynamics as the transfer function $G(s)$ derived in Part II;
2. the parameter values identified in Part III.B are internally consistent across both the Matlab and Simulink implementations.

Both model outputs follow the measured creep response closely: after the step increase in $F(t)$, the simulated displacement rises with nearly the same time constant and reaches the same plateau as $x(t)$, then decays back toward the baseline when the force returns to zero. The remaining discrepancies between the model curves and the data are small compared with the measurement noise and are likely due to (i) noise in $x(t)$, (ii) possible unmodeled viscoelastic effects beyond the simple three–parameter model, and (iii) small numerical errors from discretization of the continuous–time system.

The small residual differences between the transfer-function model and the Simulink ODE model arise solely from numerical solver tolerances and the fact that the ODE model integrates three coupled first-order equations, whereas the TF implementation evaluates a closed-form convolution. Because the underlying dynamics are analytically equivalent, both models converge to the same response when solver tolerances are tightened; any remaining mismatch between $x_{\text{pred}}(t)$ and $x_{\text{sim}}(t)$ is therefore an implementation artifact rather than a modeling error.

3 Appendix

3.1 Project A

3.1.1 Part I

Below are the possible missing items from the report.

The default parameter set (case (a)) produces a smooth, well-damped second-order response. The joint angle increases monotonically and settles at approximately 0.5 rad with no overshoot or oscillation, corresponding to a physiologically realistic reflex loop with sufficient damping and moderate feedback gain.

In case (b), increasing the feedback gain to $\beta = 200$ makes the reflex loop more aggressive. The rise time decreases and a lightly underdamped oscillation appears before $\theta(t)$ settles, indicating that the loop is operating with reduced phase margin.

Case (c) sets $\beta = 0$, so there is no feedback from the stretch receptors and the model operates effectively in open loop. The limb angle then increases almost linearly and exceeds $\pi/4$ rad well before 1 s, because nothing opposes the applied load except the passive elements. This behavior is clearly non-physiological.

Case (d) increases the reflex delay to $T_d = 0.04$ s while keeping all other parameters at default. The longer delay slows the response and slightly increases overshoot, consistent with the intuition that additional delay reduces phase margin and tends to move the system closer to oscillation.

In case (e), doubling the muscle stiffness to $k = 100$ N m makes the limb mechanically stiffer. For a given load, the steady-state angle is approximately

$$\theta_{\text{ss}} \approx \frac{M_x}{k},$$

so a larger k yields a slightly smaller steady-state θ_{ss} and a somewhat faster rise, while the response remains well damped.

Overall, we observe that:

- Increasing β (case (b)) increases responsiveness but introduces oscillations and overshoot; too large a gain would destabilize the loop.
- Removing feedback ($\beta = 0$, case (c)) leads to an effectively unbounded increase in angle under constant load, which is unrealistic.
- Increasing T_d (case (d)) tends to slow the response and promote overshoot or oscillations by reducing phase margin.
- Increasing k (case (e)) increases stiffness, decreases the steady-state angle, and modestly speeds the transient, improving static stability at the cost of a stiffer limb.

Some parameter combinations, particularly the open-loop case $\beta = 0$ and large β, produce unrealistic behavior:

- Open-loop case ($\beta = 0$): With no reflex feedback, the limb angle grows almost linearly with time under constant load, exceeding $\pi/4$ radians and ignoring anatomical joint limits and passive tissue resistance.
- High-gain case ($\beta = 200$): The oscillatory behavior is qualitatively similar to clonus-like responses but is exaggerated compared to normal physiology.

To prevent such unrealistic excursions, the model could be improved by

- adding passive joint limits that saturate $\theta(t)$ near anatomical extremes,
- introducing nonlinear stiffness/damping that increases with $|\theta|$ and $|\dot{\theta}|$, and
- incorporating supraspinal control pathways that dynamically regulate the effective reflex gain β.

To estimate the closed-loop frequency response when an analytical expression is unavailable, we drive the model with sinusoidal external torque inputs at frequencies $f = 1, 2, 3, 4, 5$ Hz,

$$M_x(t) = M_{\text{amp}} \sin(2\pi f t),$$

using the same default parameter set as in case (a). For each frequency, the Simulink model is run long enough to reach steady state. The script `projectA_partI_freqresp.m` loads the recorded time histories $M_x(t)$ and $\theta(t)$, discards at least three periods to remove transients, and then estimates gain and phase from the steady-state portion:

- The input and output amplitudes are computed from their peak-to-peak values,

  $$A_{M_x} = \tfrac{1}{2}\big(\max M_x - \min M_x\big), \qquad A_\theta = \tfrac{1}{2}\big(\max \theta - \min \theta\big),$$

  and the gain is defined as $G(f) = A_\theta / A_{M_x}$.
- The time lag Δt between the first major peak in $M_x(t)$ and the corresponding peak in $\theta(t)$ is measured, and the phase shift is computed as

  $$\phi(f) = -\,2\pi f\, \Delta t,$$

  where the minus sign reflects that a positive time lag corresponds to an output that lags behind the input.

Plotting $G(f)$ and $\phi(f)$ versus frequency yields an empirical Bode-style characterization of the closed-loop neuromuscular system. At low frequencies the gain is relatively high and the phase lag is small, consistent with effective quasi-static postural control. As frequency increases, the gain decreases and the phase lag grows, reflecting the limited bandwidth imposed by the limb inertia, muscle dynamics, and neural transmission delay.

3.2 Project B

3.2.1 Part III

Table 2: Summary statistics of the estimated viscoelastic parameters over 20 optimization runs. The coefficient of variation (`COV`) is expressed as a percentage.

Parameter	Mean estimate	Standard deviation	COV (%)
k	3.000×10^{-3}	6.520×10^{-11}	2.173×10^{-6}
k_1	6.882×10^{-2}	3.639×10^{-3}	5.289×10^{-2}
b	5.557×10^{-4}	3.092×10^{-7}	5.563×10^{-3}

References

[1] Khoo, M. C. K. (2000). *Physiological Control Systems: Analysis, Simulation, and Estimation*. Prentice Hall.

[2] Winter, D. A. (2009). *Biomechanics and Motor Control of Human Movement* (4th ed.). Wiley.

[3] Enoka, R. M. (2015). *Neuromechanics of Human Movement* (5th ed.). Human Kinetics.

[4] Kandel, E. R., Schwartz, J. H., Jessell, T. M., Siegelbaum, S., & Hudspeth, A. (2013). *Principles of Neural Science* (5th ed.). McGraw–Hill.

[5] Houk, J. C. (1976). An assessment of stretch reflex function. *Progress in Brain Research*, 44, 303–314.

[6] Findley, W. N., Lai, J. S., & Onaran, K. (1976). *Creep and Relaxation of Nonlinear Viscoelastic Materials*. Dover Publications.

[7] Prochazka, A. (1989). Sensorimotor gain control: a basic strategy of motor systems? *Progress in Neurobiology*, 33(4), 281–307.

[8] Lloyd, D. P. C. (1943). Neuronal organization of the stretch reflex. *American Journal of Physiology*, 138(2), 297–317.

[9] Matthews, P. B. C. (1972). *Mammalian Muscle Receptors and Their Central Actions*. Edward Arnold.

[10] Fung, Y. C. (1993). *Biomechanics: Mechanical Properties of Living Tissues* (2nd ed.). Springer.

[11] Brogan, W. L. (1991). *Modern Control Theory* (3rd ed.). Prentice Hall.

[12] Ljung, L. (1999). *System Identification: Theory for the User* (2nd ed.). Prentice Hall.

[13] MathWorks. (2024). *Simulink User's Guide*. https://www.mathworks.com/help/simulink/

[14] Shampine, L. F., & Reichelt, M. (1997). The MATLAB ODE Suite. *SIAM Journal on Scientific Computing*, 18(1), 1–22.

[15] Wu, M. L. (2025). *Optimization Repository*. GitHub. Available at: https://github.com/Zontafor/control-systems/tree/physiological